

Java – Beautifully Broken

Fall-through is dead, and Valhalla is finally boarding

Issue #2

Fri, 02 Oct 2026

~19 min read

Level 3/5

⚡ The short version

1. **var**, text blocks and switch expressions look cosmetic, but each one hides a sharp edge. Learn the three rules that keep them safe: no **var** with `<>`, mind the closing `"""`, and let **switch** be exhaustive.
2. **Correction and update: Valhalla's value objects (JEP 401) are targeted to JDK 28** as a preview. Yesterday we said they weren't targeted yet. They were, on 20 September.
3. **Oracle published what got faster in JDK 27.** `HashMap.putAll` from another `HashMap` is up to **86% faster** in one benchmark, with no code changes from you.

Version watch

25

current **LTS** (Sep 2025),
the safe target for
production

27

latest JDK, **GA September 2026** (non-LTS)

4.1.1

latest **Spring Boot** GA (20
Aug 2026); 4.2.0-M2 in
milestones

29

next planned **LTS**,
September 2027

Sources: Oracle Java SE support roadmap (updated 15 Sep 2026), spring.io, Spring Boot docs version list.

PART 1 Learn



var, text blocks and switch expressions: small features, big readability wins

CATCH-UP

~10 MIN

LEVEL 3

These three features change no architecture, but they touch almost every file you'll write. That's exactly why they're worth ten minutes. Each one removes a class of noise, and each one has a rule that, ignored, gives you a bug the old syntax never could. Yesterday's map showed them in passing. Today we take each apart.

Feature	Final in	JEP	Removes
<code>var</code> for local variables	10	286	Repeating a type the reader can already see
<code>var</code> in lambda parameters	11	323	The choice between annotations and short lambdas
Switch expressions (<code>-></code> , <code>yield</code>)	14	361	Fall-through bugs and "assign in every branch"
Text blocks (<code>"""</code>)	15	378	+ <code>"\n"</code> + chains and escaped quotes

All four are in every LTS from 17 onward, so if you're on 17, 21 or 25 you can use them today. (Oracle, language changes by release)

1 · **VAR**: INFERENCE, NOT DYNAMIC TYPING

var asks the compiler to write the type for you. It does not make Java dynamic. The variable still has one static type, fixed at compile time, and the class file records that type exactly as if you'd written it. **var** is allowed for locals with an initializer, loop variables, try-with-resources and (since 11) lambda parameters. It is not allowed for fields, method parameters or return types. (Oracle docs)

The OpenJDK team published a style guide for it, and its first principle is the one to remember: **reading trumps writing**. Use **var** when the initializer already tells the reader the type, like a constructor or a well-named factory method. Skip it when the type is the information. (OpenJDK LVTI style guide)

var: naive vs correct

Naive: compiles, then surprises

```
// Diamond + var: infers Object!
var queue = new PriorityQueue<>();
queue.add("order-42");
queue.add(42); // compiles fine...
// ...ClassCastException at runtime

var total = 0; // int, not long
total += bigInvoice; // silently narrows

var result = service.process(req);
// Result? Optional? Future? Who knows.
```

Correct: type on the right

```
// Put the type argument on the right
var queue = new PriorityQueue<String>();
queue.add("order-42");
queue.add(42); // compile error. Good.

var total = 0L; // say long, get long
total += bigInvoice;

PaymentResult result =
    service.process(req); // type matters
```

We compiled and ran the left pane on JDK 21: the second `add` throws `ClassCastException` from inside `PriorityQueue`. `total += bigInvoice` compiles because compound assignment includes an implicit cast. The pitfalls are guidelines G6 and G7 in the OpenJDK style guide.

Three habits cover most of the guide:

- **Never pair `var` with `<>`**. With nothing on either side to infer from, the type argument becomes `Object`.
- **Watch numeric literals**. `var x = 0` is an `int`. Use `0L`, `0.0` or an explicit type when you mean something else.
- **Keep scopes small and names descriptive**. `var customers` in a 6-line method is clearer than `List<Customer> list` in a 60-line one. (OpenJDK LVTI style guide)

2 · **TEXT BLOCKS**: THE COMPILER EATS YOUR INDENTATION (ON PURPOSE)

A text block is just a `String` constant with nicer syntax. We checked with `javap`: it compiles to an ordinary `ldc` of a string in the constant pool, so there's zero runtime cost, and it's interned like any literal. What's new is how the compiler turns your source lines into that string. It runs three steps, in this order:

How javac turns a text block into a String

STEP 1

Normalize lines

CRLF / CR become `\n`

STEP 2

Strip whitespace

common indent + trailing

STEP 3

Interpret escapes

`\n`, `\t`, `\s`, `\<newline>`

Because escapes are processed *after* stripping, `\s` (a space) survives at the end of a line, and `\` at the end of a line joins it to the next.

Source: Oracle, Programmer's Guide to Text Blocks.

The closing `"""` sets the left margin. The compiler finds the least-indented line (and the closing delimiter's line counts) and removes that much from every line. Put the closing delimiter at column 0 and you keep all the leading spaces. Trailing spaces on each line are always removed, so an editor that trims whitespace can't silently change your string. (Oracle guide)

```
String sql = """
    SELECT id, total
    FROM orders
    WHERE city = ?
    """;          // == "SELECT id, total\nFROM orders\nWHERE city = ?\n"

String msg = """
    Dear %s, your order %d \
    has shipped.\s""".formatted(name, id); // one line, trailing space kept
```

Oracle's guide adds two style rules worth adopting. **Don't use a text block for a one-line string**, and **don't mix tabs and spaces** in the indentation: each whitespace character counts as one, so a tab isn't eight spaces. -

`Xlint:text-blocks` warns you about it. (Oracle guide)

3 · SWITCH EXPRESSIONS: SWITCH THAT RETURNS A VALUE

The arrow form (`case X ->`) never falls through, and a `switch` used as an expression must cover every case. Those two rules kill the two classic `switch` bugs: the forgotten `break`, and the variable that one branch forgot to assign. Oracle's docs recommend the arrow form for new code. (Oracle docs)

A few rules people trip over:

- `yield`, not `return`, gives a block its value. `return` inside a switch expression is a compile error, because it would leave the whole method. (Oracle docs)
- Leave out `default` on enum switches when you can. If you cover every constant, the compiler adds a hidden one for you, and a new constant becomes a build error instead of a silent no-op. (Oracle docs)
- The arrow form also works as a plain statement. You don't need to produce a value to get rid of fall-through.

This is the same exhaustiveness check that makes sealed types and pattern matching so useful. That's the topic a few issues from now.

Shipping fee: statement vs expression

Java 8: statement + fall-through

```
int fee;
switch (tier) {
    case FREE:
        fee = 99;
    case PLUS:           // no break above!
        fee = 0;         // FREE gets 0 too
        break;
    case PRO:
        fee = 0;
        break;
    default:
        throw new IllegalStateException();
}
```

Java 14+: expression, no fall-through

```
int fee = switch (tier) {
    case FREE      -> 99;
    case PLUS, PRO -> 0;
    case ENTERPRISE -> {
        log.info("contract pricing");
        yield contract.fee();
    }
}; // no default: the compiler checks
// every enum constant is covered
```

Illustrative. On the left, a missing `break` makes free-tier customers ship for free, and nothing warns you. On the right, `case PLUS, PRO` groups labels, and `yield` returns a value from a block. Forget a constant and the code doesn't compile.

In the real world: the JDK team's own rules for `var`

When `var` shipped in Java 10, the OpenJDK team didn't just add the keyword. Stuart Marks published a **style guide with four principles and seven guidelines**, because the team expected exactly the misuse we showed above. Its core point: code is read far more than it's written, and often **outside an IDE** (in code review, patches and docs), where nobody can hover to see a type. So `var` is right when the name and initializer carry the meaning, and wrong when they don't. (OpenJDK LVTI style guide) It's a short read and a good thing to link from your team's code-review checklist.

Gotcha: the enum constant you didn't compile against

Exhaustive enum switches are checked at **compile** time. Say a shared library adds `Status.REFUNDED` and you upgrade the jar **without recompiling** your service. Your `switch` hits a constant it has never heard of. We tried it on JDK 21: the hidden `default` the compiler inserted throws `java.lang.MatchException`. Older releases (14–20) threw `IncompatibleClassChangeError` instead. The old switch *statement* would have silently skipped all branches. A loud failure is better than a silent one, but it means **an enum in a shared library is part of its public contract**. Recompile when you upgrade, and treat new constants as a breaking change.

Under the hood

- **`var` leaves no trace.** The class file stores the inferred type, so `var list = new ArrayList<String>()` compiles exactly like `ArrayList<String> list = ...`. It's not even a keyword: it's a *reserved type name*, so `var var = 1;` compiles, but a class named `var` doesn't. (Oracle docs)

- **Enum switches use a lookup table.** `javap` shows `javac` generating a synthetic class with a `$SwitchMap$Status` `int` array that maps each constant's `ordinal()` to a case. That indirection is why reordering an enum doesn't break compiled switches.
- **String switches hash first.** `javac` calls `hashCode()`, jumps to a bucket, then confirms with `equals()`. That's also why `switch` on a `null` `String` throws `NullPointerException` (unless you add `case null`, which needs Java 21's pattern switch).

Try it (15 minutes, any JDK 17+)

1. In `jshell`, run `var q = new java.util.PriorityQueue<>(); q.add("a"); q.add(1);` and read the exception. Then change it to `PriorityQueue<String>` and watch the second `add` become a compile error.
2. Paste a text block with the closing `"""` indented, then again with it at column 0. Compare `.length()` of the two.
3. The separate-compilation trap. Make `Status.java` (`enum Status { NEW, PAID }`) and a `Main.java` with an arrow switch over both constants and no `default`. Compile both and run. Then add `REFUNDED`, recompile **only** `Status.java`, and run `Main` again (have its `main` loop over `Status.values()`). You'll get the `MatchException` from the Gotcha box.

WHY IT MATTERS TO YOU



Day-to-day impact



Interview value



Act-now urgency

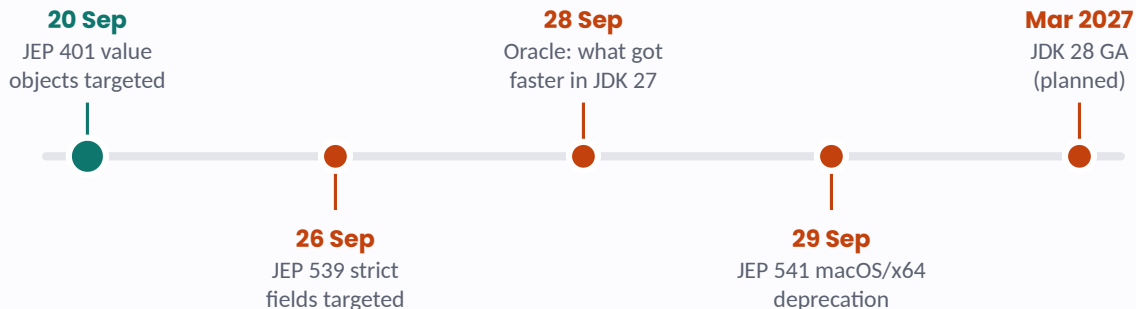


GO DEEPER

- **Switch Expressions** (Oracle docs). Arrow labels, `yield` and exhaustiveness, with examples.
- **Programmer's Guide to Text Blocks** (Oracle docs). The whitespace algorithm, new escapes and style guidelines.
- **Local Variable Type Inference: Style Guidelines** (OpenJDK). The principles behind good and bad `var`.

~10 min • A quieter week for releases, but JDK 28 got a lot more concrete. Valhalla is in, structured concurrency may finally go final, and Intel Macs are on notice.

JDK 28 takes shape (last two weeks)



Sources: [inside.java](#) announcement feed, [InfoWorld](#), 24 Sep.

Top story 1 • Update: Valhalla's value objects are coming to JDK 28

BIG DEAL

PREVIEW IN 28

What happened. Oracle's Java team announced that **JEP 401, Value Objects (Preview)**, is targeted to JDK 28.

([inside.java](#)) Its companion, **JEP 539, Strict Field Initialization in the JVM (Preview)**, was targeted on 26

September. ([inside.java](#)) A correction: yesterday's issue said value classes weren't officially targeted. They had been since 20 September. Sorry about that.

JEP 401 adds a **value modifier** for classes and records whose instances have **no identity**, only their field values. Fields are implicitly final, and `==` compares two value objects by class and field values. ([InfoQ](#))

Identity object vs value object

Today: every object has identity

```
record Point(int x, int y) {}

var a = new Point(3, 4);
var b = new Point(3, 4);

a == b;           // false: two objects
a.equals(b);     // true
synchronized (a) { ... } // allowed
```

JDK 28 preview: value record

```
value record Point(int x, int y) {}

var a = new Point(3, 4);
var b = new Point(3, 4);

a == b;           // true: same values
a.equals(b);     // still true
synchronized (a) { ... } // not allowed
```

Preview only: compile and run with `--enable-preview` on JDK 28. Syntax and semantics as described by [InfoQ's JEP 401 write-up](#).

Why it matters. Without identity, the JVM is free to **scalarize** a value object (keep its fields in registers) or **flatten** it into the fields and arrays that hold it, instead of allocating a separate heap object behind a pointer. With preview enabled, the JDK also treats classes like `Integer` and `LocalDate` as value classes. ([InfoQ](#)) JEP 539

is the JVM-level groundwork: it lets the verifier insist that certain fields are set before a constructor finishes.

(InfoQ)

The JEP is explicit that this is **not an invitation to drop** `equals`. And there are new failure modes: synchronizing on a value object or making a `WeakReference` to one fails, which matters if old code synchronizes on `Integer` instances. (InfoQ)

👁️ **What to do.** Nothing in production: this is a preview in a non-LTS release. Two cheap moves now: **grep for** `synchronized on boxed types or LocalDate`, which was already discouraged and becomes an error under the preview. And when JDK 28 early-access builds include the feature, try a `value record` for a hot small type (money, coordinates) and measure allocation with JFR.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Interview value



Top story 2 · What got faster in JDK 27, with the receipts

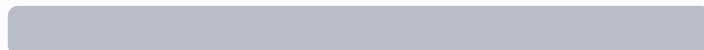
PERFORMANCE

FREE WINS

What happened. On 28 September, Per-Åke Minborg and Claes Redestad of Oracle published a roundup of **JDK 27's performance changes** across libraries, GC, the JIT compiler and runtime, each linked to its JBS issue. (inside.java) The one most Spring apps will feel: `HashMap.putAll()` and the `HashMap(Map)` constructor got a fast path when the source is itself a `HashMap` or an unmodifiable map backed by one (JDK-8371656).

HashMap(Map) copy of a 150-entry unmodifiable map, ns/op

Before the change



≈10,593 ns

JDK 27



≈1,533 ns

One JMH benchmark on AWS Graviton, from the pull request, as reported by inside.java. The gain ranged from 61% (5 entries) to 86% (this case). The authors stress these are **local effects**, not whole-application speed-ups.

The rest of the list, briefly (inside.java):

- **Crypto:** AES/ECB throughput up about 37% on one Intel CPU, SHA-3 up 16–39% with AVX2, and ML-KEM up 9–17%.
- **GC:** fixes for pathological cases, including **G1 pauses of 90+ seconds** caused by per-region code-root sets, and a Parallel GC feedback loop that caused repeated Full GCs.
- **I/O:** a redundant `fstat` call removed when opening files on Unix, about 14% faster in a stress benchmark.
- **Runtime:** compact object headers on by default (12-byte headers become 8), plus an object-monitor table now on by default, with a fix for a 30% regression in synchronization-heavy code.

Why it matters. None of these need a code change. That's the quiet argument for staying close to current JDKs: you collect dozens of small wins like these every six months. It also shows why **micro-benchmarks need context**. A 7× faster map copy matters only if map copying shows up in your profile.



Top story 3 · JDK 28's other headlines: final structured concurrency, goodbye Intel Mac

WATCH

PLATFORM

What happened. Beyond Valhalla, JDK 28's feature list is filling up. Here's where the JEPs we're tracking stand, per the latest reports:

JEP	What	Status (as reported)
543	Structured Concurrency, final ("without further change")	Proposed; implementation PR open
541	Deprecate the macOS/x64 port for removal	Targeted (29 Sep)
540	Simple JSON API (Incubator)	Targeted
544	Ahead-of-Time Code Compilation	Proposed to Target (review closed 28 Sep)

Sources: [JVM Weekly](#), [JEP 543 implementation PR](#), [inside.java](#), [InfoWorld](#), [InfoQ](#).

Structured concurrency has been a preview through seven rounds (JDK 27 had the seventh, JEP 533). JEP 543 proposes finalizing it in JDK 28 without further change. ([JVM Weekly](#)) The implementation pull request by Alan Bateman removes the preview annotations and bumps `@since` to 28. It is still in review. ([GitHub](#))

macOS/x64: Oracle says Apple has moved its hardware to AArch64, so its engineers **stop maintaining the Intel Mac port as of JDK 27**. The port isn't removed yet; JEP 541 marks it for removal in a future release.

(OSnews)

Why it matters. Once structured concurrency is final, you can use it without `--enable-preview`. It will likely reach most teams through the next LTS, Java 29 in September 2027. The Mac change is about **build agents and laptops**. Intel Mac CI runners and older developer machines will eventually need an Apple Silicon or Linux replacement. JDK 25 LTS keeps working on them in the meantime.

👁️ What to do. Check whether any **CI pipeline or release build still targets macOS/x64**, and plan its replacement before JDK 29. If you've been waiting to adopt structured concurrency, its API has stabilized: prototype with JDK 27's preview now.



⚡ Quick hits



Post-quantum crypto got much faster.

Oracle reports ML-KEM running **3.18× faster** on JDK 28 with new CPU intrinsics, and HSS/LMS signature checks gaining from SHA-256 intrinsics. ([inside.java](#))



Apache Groovy 6.0 is GA

after three release candidates, with a lot of lexer and parser performance work. Relevant if your Gradle builds or Jenkins pipelines use Groovy. ([InfoQ](#))



TornadoVM 7.0

adds a `TileContext` API and a new `@Parallel` annotation for writing GPU kernels in Java. (InfoQ)



Open Liberty 26.0.0.10 beta

adds JDK 27 support, and warns that G1 is now the default GC in constrained environments, so GC behaviour there may change. (InfoQ)



Micrometer Tracing 1.8.0-M2 drops Zipkin.

Its second milestone deprecates the Zipkin exporter, since OpenTelemetry discontinued it. If you trace via Zipkin, plan a move to OTLP. (InfoQ)



Azul Payara 7.4.0

fixes CVE-2026-68497 in Jackson Databind and removes MicroProfile OpenTracing support. (InfoQ)



Coming up.

Maven 4.0 GA (expected "in the coming weeks"), Spring Boot 4.2 GA (no date yet), Jakarta EE 12 Core Profile (before 1 December), and JDK 28 in March 2027.



Revision card

- **var** is static typing. The compiler writes the type; the class file records it. Locals only: no fields, parameters or return types.
- Never `var x = new Foo<>()`. The diamond infers `Object`. Put the type argument on the right.
- `var n = 0` is an `int`. Use `0L` or an explicit type when you mean `long`.
- Text blocks: the closing `"""` sets the margin. Trailing spaces are stripped; use `\s` to keep one and `\` to join lines.
- Arrow **case** never falls through. Switch expressions must be exhaustive, and blocks return with `yield`, not `return`.
- No **default** on enum switches turns a new constant into a compile error. Unrecompiled code gets a `MatchException`.
- **JDK 28 (Mar 2027)**: value objects (JEP 401) in preview, structured concurrency proposed final (JEP 543), Intel Mac port deprecated (JEP 541).
- **JDK 27 sped up HashMap copies** by 61–86% in Oracle's benchmarks. Free, but local: profile first.

Java – Beautifully Broken is researched and written by Claude every morning from OpenJDK, Oracle and Spring announcements, official docs, InfoQ and engineering blogs. Vendor numbers are labelled as claims, and every item links to its source. Want a topic sooner or a different depth? Change `java/settings.yml` or just reply. ☕